InfoPharma Ordering System (IPOS)

Contents

User Stories IPOS – SA System 2
Prioritization Table of User Stories 9
Component Diagram 12
Component Diagram Detailed 13
Design Class Diagram 14
Generated Java Code 14
Test Cases 17
Hardware Requirements: 22

User Stories IPOS – SA System

Overview of the User Stories: All of the user stories below are related to the IPOS-SA system and gathered from the required functionality provided in the Brief. Each User Story follows the Industry standard User Story structure of Title -> Description (Optional) -> Acceptance Criteria. All of these User Stories are created from the Users perspective when using the system. Furthermore, the Acceptance Criteria has been made in a way that it is either a pass or fail metric for each point. The main goal of these user stories was to provide a easy transition to turning these user stories into Tickets (tasks to complete) which can be done within a Sprint (2 weeks) and can be assigned to a developer on a platform such as GitHub Issues or JIRA. Also, each Epic of the System has been broken down into the User Stories as shown below. Finally, each user story has met the INVEST framework.

Epic: User Account Management (IPOS – SA – ACC)

Description: This epic covers the creation, configuration, and access levels of different user roles within the IPOS-SA system to ensure secure and tailored access for merchants, administrators, and managers.

User Story 1: Merchant Account Creation:

- **Title: As an Administrator, I want to create a new Merchant account which requires contact and financial details so that the account is properly configured before it becomes active.**

- **Description: Specific metadata is required to create a Merchant Account (Contact details, Credit Limit, and Discount Plan for example).**
- **Acceptance Criteria (Must be yes or no):**
    - **System must provide fields for Contact Details, Credit Limit, and Discount Plan selection.**
    - **The account must remain "Inactive" and cannot be created if any of the mandatory fields are left blank.**
    - **Once all details are saved, the account status should update to "Active."**

User Story 2: Fixed Discount Plan Assignment:

- **Title: As an Administrator, I want to assign a fixed discount rate to a Merchant so that a consistent discount is applied to every order they place.**
- **Acceptance Criteria:**
    - **The system must allow the user (Administrator) to input a specific percentage rate for the "Fixed" plan.**
    - **The system must automatically reduce the "Amount Due" for every order placed by this merchant based on that fixed rate.**

User Story 3: Flexible Discount Plan Configuration:

- **Title: As a Manager, I want to set up a flexible discount plan based on monthly spend so that merchants have a incentive to spend more.**
- **Acceptance Criteria:**
    - **The system must track the total value of all orders within a single calendar month.**
    - **The system must apply tiered rates (e.g. 1% for <£1k, 2% for £1k-£2k, 3% for >£2k).**
    - **At the end of the month, the system must calculate the total discount to be either refunded via check or credited to the next order.**

User Story 4: Role-Based Access Control (RBAC):

- **Title: As an Administrator, I want to assign specific roles (Merchant, Admin, Manager) to users so that they can only access the relevant functionality that they require.**
- **Acceptance Criteria:**
    - **Merchant accounts must only have access to IPOS-SA-CAT (Catalogue) and IPOS-SA-ORD (Orders).**
    - **Manager accounts must have access to IPOS-SA-RPRT (Reporting) and Merchant account settings.**
    - **Administrator accounts must have full access to all packages, including IPOS-SA-ACC.**

User Story 5: Managing Defaulted Accounts:

- **Title: As a Manager, I want to manually restore a merchant's account from 'In Default' status so that they can access the functionality of their account again after a long-term payment issue (e.g longer than 30 days without payment) is resolved.**
- **Acceptance Criteria:**
    - **The system must restrict the "Restore" action for "In Default" accounts (delays >30 days) to Manager-level users only.**
    - **A Manager must be able to change the status from "In Default" to either "Normal" or "Suspended."**
    - **The system must log which Manager performed the status change.**

User Story 6: Managing Accounts:

- **Title: As a Manager, I want to be able to access merchant accounts and alter their credit limits, discount plans.**
- **Acceptance Criteria:**
    - **The system must allow a manager to access merchant accounts.**
    - **The system must allow a manager to alter the credit limits and discount plans of the merchant accounts (once logged in).**

Epic: Electronic Catalogue and Inventory Control

Description: This epic enables InfoPharma to maintain an up-to-date electronic catalogue of pharmaceutical products, manage stock levels, and monitor inventory thresholds to ensure products are available to the customers.

User Story 1: Catalogue Initialization

- **Title: As an Administrator, I want to create a new electronic catalogue (Database) so that pharmaceutical products can be stored and managed in the system.**
- **Acceptance Criteria:**
    - **The system must allow the creation of a new catalogue only if one does not exist already.**
    - **The system must initialize the catalogue with the appropriate fields such as Product ID, Description, Unit Price, and Availability with the correct data types (e.g int, String).**

User Story 2: Product Creation

- **Title: As an Administrator, I want to add new products to the catalogue so that they are available for merchants to order.**
- **Description: Requires Product ID, Description, Unit Price, and Availability.**
- **Acceptance Criteria:**
    - **The system must enforce a unique Product ID for every new entry.**
    - **The system must validate that the Unit Price is greater than zero.**

    - **The system must validate that Availability is not a negative value.**

User Story 3: Product Discontinuation/Removal

- **Title: As an Administrator, I want to remove products from the catalogue so that merchants cannot order discontinued or out of stock items.**
- **Acceptance Criteria:**
    - **The system must prompt for a confirmation "Yes/No" before a product is deleted. (This is to prevent accidental deletion.)**
    - **The system must log the deletion activity, including which user performed the action.**

User Story 4: Product Data Maintenance

- **Title: As an Administrator, I want to update existing product details so that the catalogue information remains up to date and accurate for merchants.**
- **Acceptance Criteria:**
    - **The system must prevent the modification of an existing Product ID.**
    - **The system must validate that any updated price or stock level (Availability) remains non-negative.**

User Story 5: Internal Product Search

- **Title: As an Administrator, I want to search for products using multiple criteria so that I can quickly locate products and see all of the information relating to the product.**
- **Acceptance Criteria:**
    - **The system must allow searching by Product ID, Description, or Price Range.**
    - **The system must display a "No products found" message if no matches exist.**

User Story 6: Merchant Catalogue Browsing

- **Title: As a Merchant, I want to search and view the product catalogue so that I can identify items I wish to purchase.**
- **Acceptance Criteria:**
    - **The system must provide "Read-Only" access, the merchant cannot edit any catalogue details.**
    - **The system must hide the products stock levels from the merchant view (only showing "Available" or "Out of Stock").**
    - **The system must allow for products to be searched from incomplete words (para -> can search for paracetamol)**

User Story 7: Stock Delivery Recording

- **Title: As an Administrator, I want to record new stock deliveries so that inventory levels are automatically increased.**
- **Acceptance Criteria:**
    - **The system must record the delivery date and the quantity received.**

    - **The system must automatically increment the "Availability" of the product by the delivery quantity.**
    - **The system must reject any delivery quantity equal to or less than zero.**

User Story 8: Minimum Stock Thresholds

- **Title: As an Administrator, I want to set minimum stock thresholds for each item so that the system can flag when a product reaches low enough levels of stock.**
- **Acceptance Criteria:**
    - **The system must allow a specific minimum stock threshold value to be assigned to each unique Product ID.**
    - **The system must validate that threshold values are not negative.**

User Story 9: Automated Low-Stock Warnings

- **Title: As an Administrator, I want to receive automated UI warnings when stock falls below thresholds set so that I can trigger the re-ordering process to replenish the stock.**
- **Acceptance Criteria:**
    - **The system must automatically flag a product in the UI when Current Availability < Threshold.**
    - **The warning must include the Product ID, Description and current stock level.**

User Story 10: Low-Stock Reporting

- **Title: As a Manager, I want to generate a low-stock report so that I can oversee inventory trends and plan supplier orders.**
- **Acceptance Criteria:**
    - **The system must generate a report listing all products currently below their defined thresholds.**
    - **The report must reflect real-time catalogue data at the moment of generation.**
    - **The report must be accessible through the IPOS-SA-RPRT package.**

Epic: Order Fulfillment and Financial Tracking

Description: This epic enables merchants to place and track orders while providing InfoPharma with the tools to manage stock reduction, invoice generation and payment recording.

User Story 1: Placing a New Order

- **Title: As a Merchant, I want to place an order by selecting items from the current catalogue so that I can restock my pharmacy.**
- **Description: Dependency: Requires integration with the catalogue (IPOS-SA-CAT).**
- **Acceptance Criteria:**
    - **The system must verify the merchant is authorized and authenticated before allowing order entry (That they have a Merchant Account).**

    - **The merchant must be able to select one or multiple items and specify valid quantities.**
    - **Upon submission, the order must be successfully recorded in the database with an initial status (e.g "Accepted").**

User Story 2: Real-time Order Tracking

- **Title: As a Merchant, I want to track the progress of my submitted orders so that I can monitor when they are processed or dispatched.**
- **Acceptance Criteria:**
    - **The merchant must see a list of their orders with current statuses (e.g "Accepted" "Processing" "Dispatched").**
    - **Status updates must be reflected in the UI as they are updated by staff or automated processes.**

User Story 3: Financial Balance Oversight

- **Title: As a Merchant, I want to view my current outstanding balance so that I know the total amount I need to repay InfoPharma.**
- **Acceptance Criteria:**
    - **The system must display a read-only total of all unpaid invoices for the merchant.**
    - **The balance must automatically update when new invoices are raised or payments are recorded.**
    - **The system must display amount of time elapsed since payment has been due.**

User Story 4: Automated Stock Reduction

- **Title: As a Merchant, I want the system to automatically reduce product availability upon order acceptance so that I do not accidentally order items that have already been sold to others.**
- **Description: This story ensures the inventory data remains synchronized with successful sales transactions without manual intervention.**
- **Acceptance Criteria:**
    - **Stock levels for specific items must be decremented only *after* the order status moves to "Accepted"**
    - **The reduction quantity must exactly match the quantity specified in the accepted order.**

User Story 5: Automated Invoice Generation

- **Title: As an Administrator, I want the system to generate an invoice for every accepted order so that billing and payments can be tracked.**
- **Acceptance Criteria:**
    - **The system must automatically generate an invoice once an order is marked as "Accepted"**
    - **The invoice must include Merchant contact details, order items, VAT number, and final total values.**

    - **Invoices must be stored in the system database for future reference.**

User Story 6: Recording Merchant Payments

- **Title: As an Administrator, I want to manually record payments against existing invoices so that outstanding balances are kept up to date.**
- **Acceptance Criteria:**
    - **The user must be able to record payments made via Bank Transfer, Card or Check.**
    - **The system must update the merchant's outstanding balance immediately upon saving the payment.**

Epic: Operational and Financial Reporting

Description: This epic provides management and staff with tools to analyze sales performance, merchant activity and stock turnover through on-screen visualization and printed reports.

User Story 1: Sales Turnover Reporting

- **Title: As a Manager, I want to generate a turnover report for a specific period so that I can analyse company sales performance.**
- **Acceptance Criteria:**
    - **The system must allow users to select a specific "Start Date" and "End Date."**
    - **The report must display total quantities sold and the total revenue received.**
    - **The report must be viewable on-screen and formatted for printing.**

User Story 2: Merchant History Tracking

- **Title: As a Manager, I want to generate an order history report for a specific merchant so that I can review their purchase/order history.**
- **Acceptance Criteria:**
    - **The report must include: Order ID, Order Date, Dispatch Date, Total Value and Payment Status (e.g Pending).**
    - **The system must include a "Totals" line at the end of the report to summarize the period.**

User Story 3: Individual Merchant Activity Report

- **Title: As a Manager, I want to generate a detailed activity report for a merchant so that I can review their ordering behavior.**
- **Acceptance Criteria:**
    - **The report header must display the merchant's contact details.**
    - **Each order in the list must show: individual items, quantities, item costs, total order cost and any applied discounts.**

User Story 4: Global Invoice Monitoring

- **Title: As a Manager, I want to view all invoices raised against all merchants for a given period so that I can monitor company-wide billing activity.**
- **Acceptance Criteria:**
  - **The report must list every invoice issued within the user-defined date range across the entire system.**
  - **Each entry must include the Merchant Identifier, Invoice ID, Date, Amount, and Payment Status.**

User Story 5: Stock Turnover Analysis

- **Title: As a Manager, I want to generate a stock turnover report so that I can compare products sold against products newly received.**
- **Description: Requires accurate data from both IPOS-SA-ORD and IPOS-SA-CAT ensure these systems are operational before tackling this.**
- **Acceptance Criteria:**
  - **The report must display quantities of products sold versus quantities of products received into stock for the selected period.**
  - **Data must be accurately pulled from both the IPOS-SA-ORD (sales) and IPOS-SA-CAT (deliveries) records.**

Prioritization Table of User Stories

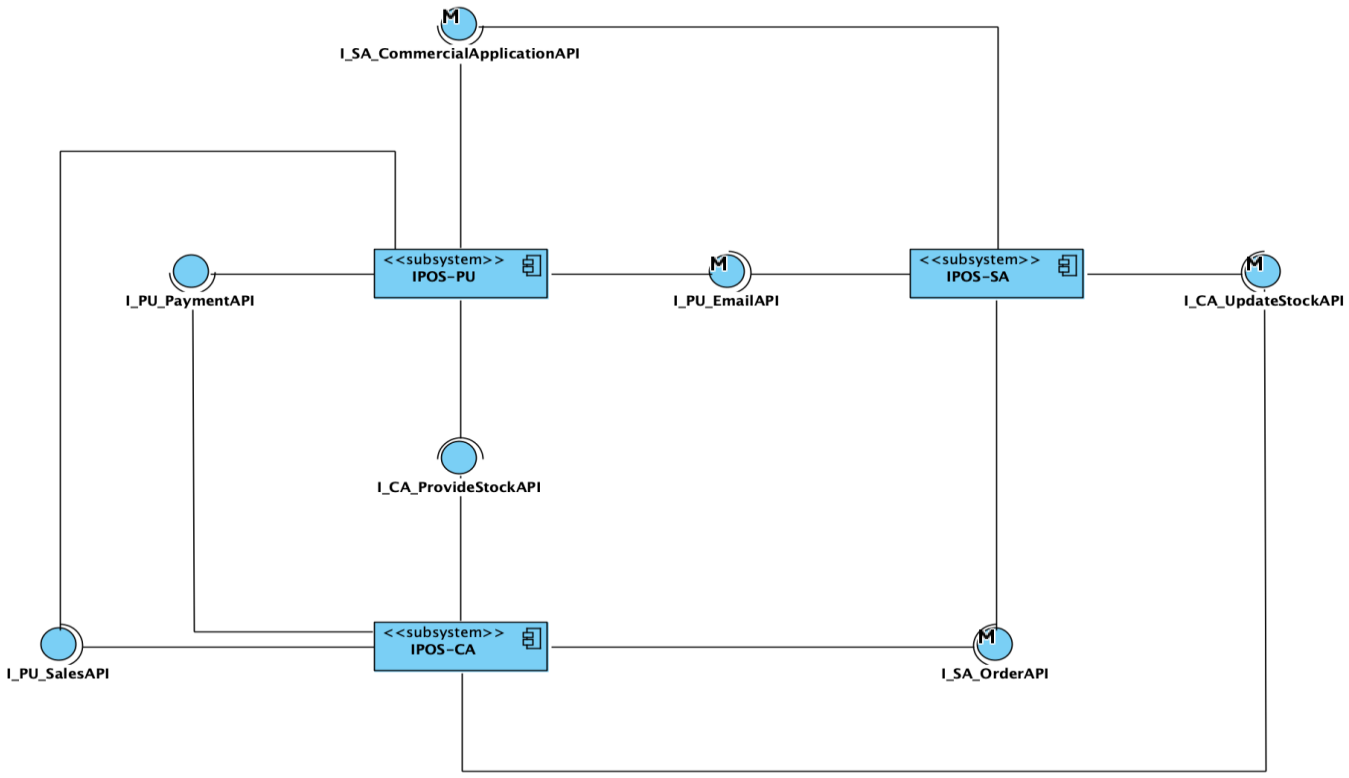
Description: Ensure that the most critical parts of the system are completed first. Also showing estimated effort and risk of each user story for the developer's information.

Index	User Story Title	Priority	Effort	Risk	Dev Impact / Notes
ACC-US1	Merchant Account Creation	**Must**	Medium	Low	Vital for system entry and configuration.
ACC-US2	Fixed Discount Plan	**Should**	Small	Low	Basic financial logic for specific merchants.
ACC-US3	Flexible Discount Plan	**Should**	Large	**High**	Complex monthly spend tracking and calculation.
ACC-US4	Role-Based Access Control	**Must**	Medium	Medium	Essential for system security and role isolation.
ACC-US5	Managing Defaulted Accounts	**Should**	Small	Low	Required for Director-level manual overrides.

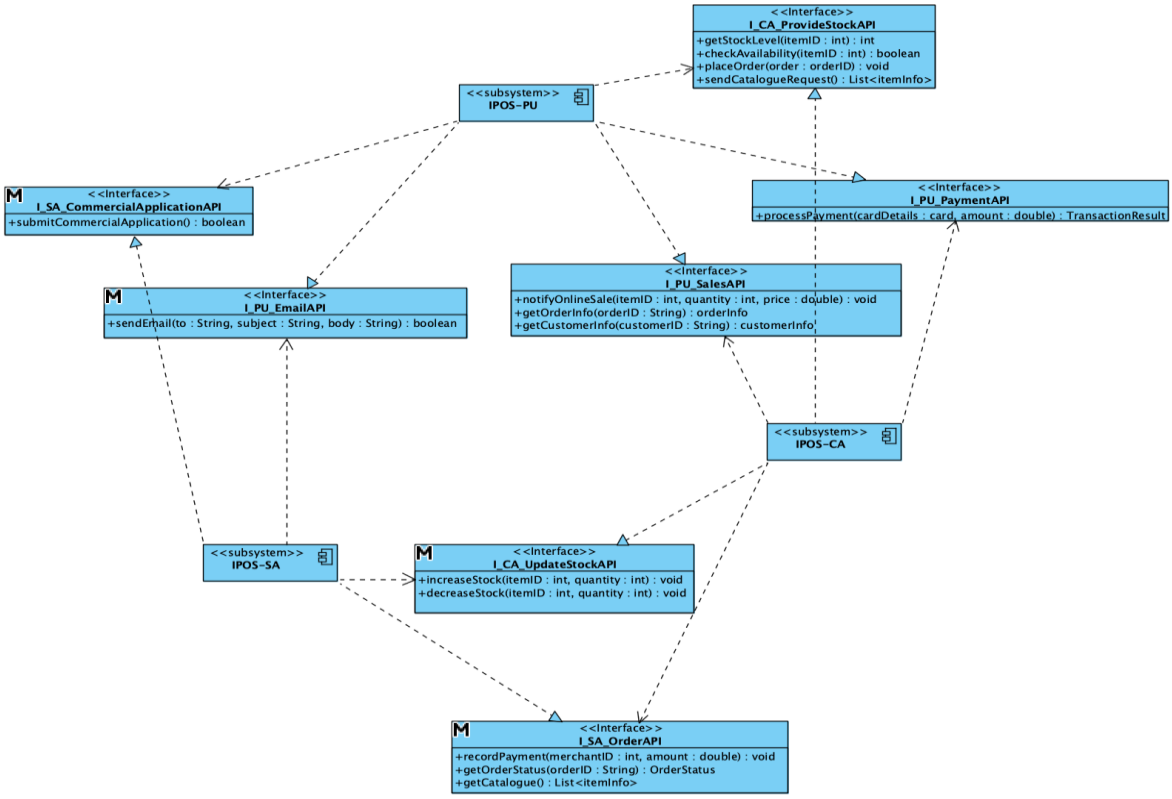
Index	User Story Title	Priority	Effort	Risk	Dev Impact / Notes
ACC-US6	Managing Accounts (Edit)	**Must**	Small	Low	Required for updating financial terms.
CAT-US1	Catalogue Initialization	**Must**	Small	Low	Foundation for the electronic database.
CAT-US2	Product Creation	**Must**	Medium	Low	Core function to populate goods.
CAT-US3	Product Discontinuation	**Should**	Small	Low	Maintenance tool for removing obsolete items.
CAT-US4	Product Data Maintenance	**Must**	Small	Low	Ensures price/description accuracy.
CAT-US5	Internal Product Search	**Should**	Medium	Low	Admin efficiency for locating stock.
CAT-US6	Merchant Browsing	**Must**	Medium	Low	Primary B2B interface for orders.
CAT-US7	Stock Delivery Recording	**Must**	Medium	Low	Direct link to physical inventory arrivals.
CAT-US8	Minimum Stock Thresholds	**Should**	Small	Low	Configuration for low-stock logic.
CAT-US9	Automated Low-Stock Warnings	**Must**	Medium	Medium	Real-time alerts for re-ordering.
CAT-US10	Low-Stock Reporting	**Should**	Medium	Low	On-demand overview of threshold breaches.
ORD-US1	Placing a New Order	**Must**	Large	**High**	High complexity; core business value.
ORD-US2	Real-time Order Tracking	**Must**	Medium	Medium	Critical for merchant fulfilment visibility.
ORD-US3	Financial Balance Oversight	**Must**	Small	Low	Critical for credit management.
ORD-US4	Automated Stock Reduction	**Must**	Medium	Medium	Essential for database integrity.
ORD-US5	Automated Invoice Generation	**Must**	Medium	Medium	Required for legal and reference storage.
ORD-US6	Recording Merchant Payments	**Must**	Medium	Medium	Manual entry of external bank transfers.
RPT-US1	Sales Turnover Reporting	**Must**	Medium	Low	Key management metric for revenue.

Index	User Story Title	Priority	Effort	Risk	Dev Impact / Notes
RPT-US2	Merchant History Tracking	**Must**	Medium	Low	Audit trail for specific client orders.
RPT-US3	Individual Merchant Activity	**Should**	Medium	Low	Granular view of items and discounts.
RPT-US4	Global Invoice Monitoring	**Should**	Medium	Low	Monitoring company-wide billing.
RPT-US5	Stock Turnover Analysis	**Must**	Large	Medium	Complex correlation of ORD and CAT data.

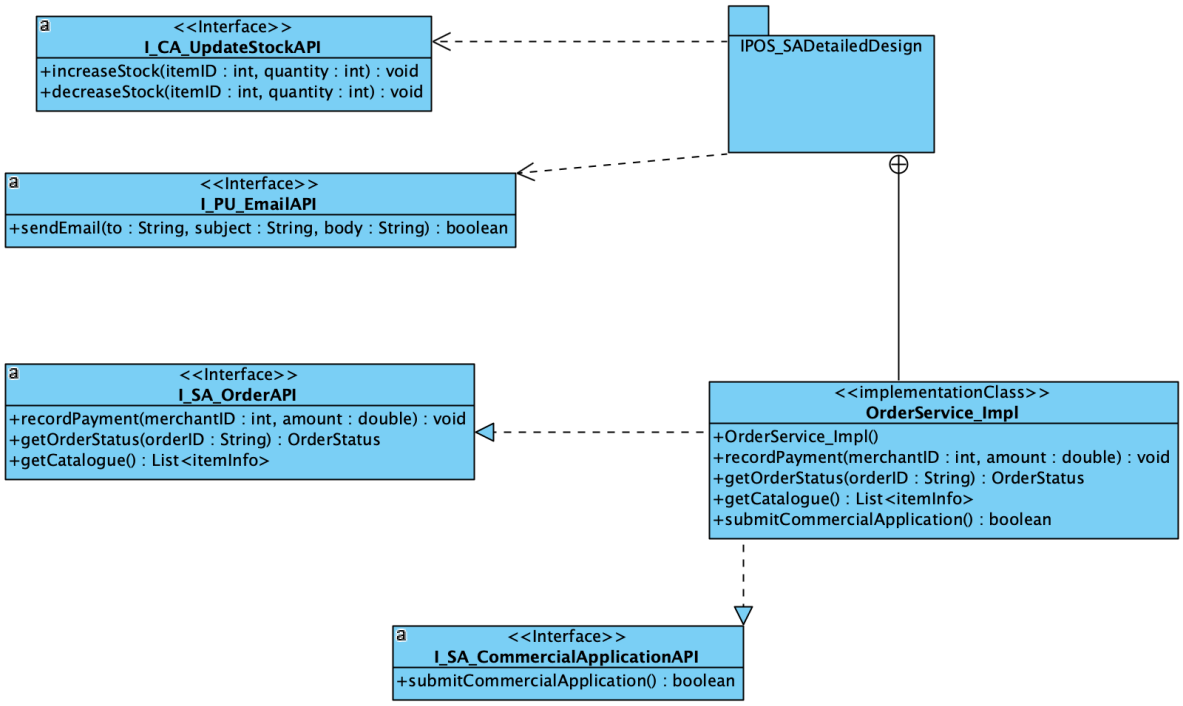
Component Diagram



Component Diagram Detailed



Design Class Diagram



Generated Java Code

```
public interface I_CA_UpdateStockAPI {

    /**
```

```
    * the increaseStock method increases the stock level of an item in the
merchant's local inventory
    * @param itemID  : the unique identification of the catalogue item that
we increase the stock of.
    * @param quantity  : the quantity we increase the stock of the respective
item by.
    */
   void increaseStock(int itemID, int quantity);

   /**
    * the decreaseStock method decreases the stock level of an item in the
merchant's local inventory
    * @param itemID  : the unique identification of the catalogue item that
we decrease the stock of
    * @param quantity  : the quantity we decrease the stock of the respective
item by.
    */
   void decreaseStock(int itemID, int quantity);

}
```

```
public interface I_PU_EmailAPI {

   /**
    * the sendEmail method sends an email via an email provider
    * @param to  : the recipient's email address.
    * @param subject  : the subject line of the email to hint at the contents
of the email.
    * @param body  : the main content of the email being sent.
    *
    */
   boolean sendEmail(String to, String subject, String body);

}
```

```
public interface I_SA_CommercialApplicationAPI {
/**
* the submitCommercialApplication method submits a commercial membership
application to the server application.
*/
   /
   boolean submitCommercialApplication();

}
public interface I_SA_OrderAPI {

   /**
    * the recordPayment method records a payment made by a merchant to
InfoPharma,
```

```
     * and updates the merchants outstanding balance accordingly.
     * @param merchantID  : the ID of the merchant that made the payment.
     * @param amount  : the amount paid by the respective merchant in the
transaction.
     */
   void recordPayment(int merchantID, double amount);

   /**
     * the getOrderStatus method returns the current status of an order.
     * the getCatalogue method returns the full catalogue of all
pharmaceutical Infopharma products, available for merchants to order from.
     * @param orderID  : the unique identifier of the order for purposes of
tracking the order, and saving order details.
     */
   OrderStatus getOrderStatus(String orderID);

   List<itemInfo> getCatalogue();

}
```

```
package IPOS_SADetailedDesign;

public class OrderService_Impl implements I_SA_OrderAPI,
I_SA_CommercialApplicationAPI {
/**
* the class OrderService_Impl implements the interface classes, and
initialises the order management services for the IPOS_SA subsystem.
*/
   /
   public OrderService_Impl() {
      // TODO - implement OrderService_Impl.OrderService_Impl
      throw new UnsupportedOperationException();
   }

   /**
     * the method recordPayment records a payment made by a merchant to
InfoPharma,
     * and updates the merchants outstanding balance accordingly.
     * @param merchantID  : the ID of the merchant that is making the payment
to InfoPharma.
     * @param amount  : the amount paid by the respective merchant.
     */
   public void recordPayment(int merchantID, double amount) {
      // TODO - implement OrderService_Impl.recordPayment
      throw new UnsupportedOperationException();
   }

   /**
     * the getOrderStatus method returns the current status of an order.
```

```
    * the getCatalogue method returns the full catalogue of all
pharmaceutical Infopharma products, available for merchants to order from.
    * the submitCommercialApplication method submits a commercial membership
application to the server application.
    * @param orderID  : the unique identification of the particular order
made, used to identify,track or save the order.
    */
   public OrderStatus getOrderStatus(String orderID) {
      // TODO - implement OrderService_Impl.getOrderStatus
      throw new UnsupportedOperationException();
   }

   public List<itemInfo> getCatalogue() {
      // TODO - implement OrderService_Impl.getCatalogue
      throw new UnsupportedOperationException();
   }

   public boolean submitCommercialApplication() {
      // TODO - implement OrderService_Impl.submitCommercialApplication
      throw new UnsupportedOperationException();
   }

}
```

Test Cases

Overview of Test Cases: To implement these in our system we will use the most common Java Testing Framework (Junit).

Method	Test ID	Parameter Value	Comment	Expected Outcome
recordPayment(merchantID : int, amount : double):void	1	merchantID = 0001 amount = 250	Merchant 101 exists in the system	Success Payment successfully recorded with no exception thrown
recordPayment(merchantID : int, amount : double):void	2	merchantID = 5002 amount = 120.75	Valid merchant with a decimal payment.	Success Payment recorded, balance updated with decimal precision
recordPayment(merchantID :	3	merchantID = -999	Negative	Failure

int, amount : double):void		amount = 500.0	merchant ID provided	Exception raised : (‘Invalid merchant ID’). No payment recorded
recordPayment(merchantID : int, amount : double):void	4	merchantID = 9a9a amount = 250	Merchant does not exist	Failure Exception raised : (‘Invalid merchant ID’). No payment recorded
recordPayment(merchantID : int, amount : double):void	5	merchantID = 0001 amount = -50	Valid merchant with negative amount	Failure Exception raised(‘Invalid payment amount’). No payment recorded
recordPayment(merchantID : int, amount : double):void	6	merchantID = 0001 amount = 0	Valid merchant. Zero for amount provided	Failure . Exception raised (‘payment amount must be greater than 0’). No payment recorded

Method	**Test ID**	**Parameter Value**	**Comment**	**Expected Outcome**
getOrderStatus(orderID : string)	7	orderID = ORD001	Order exists	Success Correct order status returned (e.g. ‘Shipped’)
getOrderStatus(orderID : string)	8	orderID = 'IP2009'	Valid order ID	Success Correct order status returned (e.g. ‘Shipped’)
getOrderStatus(orderID : string)	9	orderID = hello1	Order doesn’t exist	Failure Exception raised (‘Order not found’)
getOrderStatus(orderID : string)	10	orderID = ‘ ’	Empty order id string	Failure Exception raised (‘Invalid order ID’)

getOrderStatus(orderID : string)	11	orderID = null	Null order ID provided	Failure Exception raised (‘Order ID cannot be null’)

Method	Test ID	Parameter Value	Comment	Expected Outcome
getCatalogue() : List<ItemInfo>	12	none	Catalogue contains items	Success Non-empty List<ItemInfo> returned.
getCatalogue() : List<ItemInfo>	13	none	Catalogue exists but is empty	Success. Empty list returned (not null)

Method	Test ID	Parameter Value	Comment	Expected Outcome
submitCommercialApplication():boolean	14	none	Application data is valid	Success Returns true. Application submitted
submitCommercialApplication():boolean	15	none	Required application data missing	Failure Returns false. Application not submitted
submitCommercialApplication():boolean	16	none	Application already submitted previously	Failure Returns false. Duplicate application rejected

Method	Test ID	Parameter Value	Comment	Expected Outcome
increaseStock(itemID : int, quantity : int):void	17	itemID = 10 quantity = 5	Item exists with valid quantity	Success Stock level increases by 5

increaseStock(itemID : int, quantity : int):void	18	itemID = 15 quantity = 1	Adding a single item to a product that is out of stock	Success Stock level increases from 0 to 1.
increaseStock(itemID : int, quantity : int):void	19	itemID = 9999999 quantity = 5	Item doesn't exist	Failure. Exception raised : ('Item not found). Stock unchanged
increaseStock(itemID : int, quantity : int):void	20	itemID = 10 quantity = -5	Valid item with negative quantity provided	Failure Exception raised: ('Quantity must be positive'). Stock should remain unchanged
increaseStock(itemID : int, quantity : int):void	21	itemID = 10 quantity = 0	Valid item id, zero quantity provided	Failure Exception raised: ('Quantity must be greater than zero'). Stock remains unchanged
increaseStock(itemID : int, quantity : int):void	22	itemID = 10 quantity = 2147483647	Quantity value is too large.	Failure Exception raised: ('Quantity exceeds system limits'). Stock level unchanged.

Method	**Test ID**	**Parameter Value**	**Comment**	**Expected Outcome**
decreaseStock(itemID : int, quantity : int):void	23	itemId = 10 quantity = 3	Stock available >= 3	Success Stock decreases correctly by 3
decreaseStock(itemID : int, quantity : int):void	24	itemId = 10 quantity = 100	Stock insufficient	Failure Exception raised ('Insufficient stock'). Stock unchanged
decreaseStock(itemID : int, quantity : int) : void	25	itemId = 999999 quantity = 3	Item doesnt exist	Failure Exception raised ('Item not found'). Stock not changed

decreaseStock(itemID : int, quantity : int):void	26	itemId = 10 quantity = -2	Valid item. Negative quantity provided	Failure Exception raised (‘Quantity must be positive’). Stock not changed
decreaseStock(itemID : int, quantity : int):void	27	itemID = 10 quantity = 0	Requesting a 0 quantity stock reduction	Failure Exception raised ('Reduction quantity must be greater than zero')

Method	**Test ID**	**Parameter Value**	**Comment**	**Expected Outcome**
sendEmail(to : string, subject : string, body : string) : boolean	28	to = ‘user@gmail.com’ subject = ‘welcome’ body = ‘approved’	Valid email format and content	Success Returns true. Email sent successfully
sendEmail(to : string, subject : string, body : string): boolean	29	to = ‘user@gmail.com ’ subject = ‘Invoice#1*’ body = ‘Details attached’	Valid format containing special characters	Success Returns true with email successfully sent
sendEmail(to : string, subject : string, body : string):boolean	30	to = ‘invalidEmail’ subject = ‘Test’ body = ‘Body’	Invalid email format	Failure Returns false. Email not sent
sendEmail(to : string, subject : string, body : string):boolean	31	to = ‘user@domain…com’ subject = ‘Test’ body = ‘Body’	Invalid email format	Failure Returns false. Email not sent
sendEmail(to : string, subject : string, body : string):boolean	32	to = ‘ ’ subject = ‘Test’ body = ‘Body’	Empty email for recipient	Failure Exception raised : (‘Recipient email required’)
sendEmail(to : string, subject : string, body : string):boolean	33	to = null subject = ‘Test’ body = ‘Body’	Null email provided	Failure Exception raised : (‘Recipient email cannot be null’)
sendEmail(to : string, subject	34	to = ‘user@gmail.com’	Empty	Failure

: string, body : string): boolean		subject = ‘ ’ body = ‘ ’	subject and body	Returns false. Email not sent due to incomplete content for body and subject

Hardware Requirements:

Minimum : A Windows laptop/desktop PC or MacOS, Dual Core CPU (e.g i3 – 8100) 4GB of RAM at least 30 GB of free disk space for required programs and database servers. A display of a resolution at or above 1280 x 720.

Recommended : A Windows laptop/desktop PC or MacOS, Quad Core CPU (e.g AMD Ryzen 3 3100) 8GB of DDR4/5 RAM at least 30 GB of free disk space (SSD – solid state drive) for required programs and database servers. A display of a resolution at or above 1920 x 1080.